

CurrentCache: Dynamic Switching Between Domain-Specific Cache Controllers

Eric Hall, Xiaocen Liu, Justin Shin, Mingyao Sun

School of Electrical Engineering and Computer Science, University of Michigan



Introduction

Domain-specific caches (DSCs) offer massive improvements over scratchpad or simple dense cache algorithms for their target applications, but struggle when presented with applications outside their domain. We build upon previous work implementing high-performance caches for sparse-sparse matrix multiplication to design and implement a cache that dynamically supports sparse-sparse, sparse-dense and dense-dense matrix multiplication applications with little to no overhead compared to a prior art. SpGEMM, standing for sparse-sparse matrix multiplication is a bottleneck in GNNs, sparse transformers, scientific computing, database operations and more. We offer the following contributions:

1. CurrentCache, a hybrid DSC and simple-dense cache controller.
2. A methodology for combining multiple DSCs into the same cache.

Background and Prior Work

X-Cache [4]

X-Cache is a framework for generating DSCs providing address translation, custom tagging, table/data-structure walking, and refills or prefetching for a specialized problem by using a microcode engine as the cache controller. Notably, X-Cache does not provide any infrastructure or insight for mixing domain-specific cache controllers with a general fallback controller or each other, even though popular accelerators like GPUs and near-memory accelerators like on-chip neural engines may still benefit from the increased re-use in domain-specific caching for memory-bound applications.

SeaCache [2]

SeaCache builds upon X-Cache by implementing several optimizations for sparse matrix calculations. SeaCache either packs multiple short fibers together into a single cache line or splits a long fiber between two or more cache lines. Instead of using a virtual or physical address, SeaCache uses the coordinate of the fiber to index the cache with a fuzzy matching algorithm to support split fibers. Each element in the cache line is tagged with the coordinate along the minor axis for hit detection, since nonzero elements may be stored compactly in the same cache line. SeaCache supports a guided LFU replacement policy that tracks some uncached fibers such that these "virtual" fibers will not evict a more frequently used fiber currently in the cache.

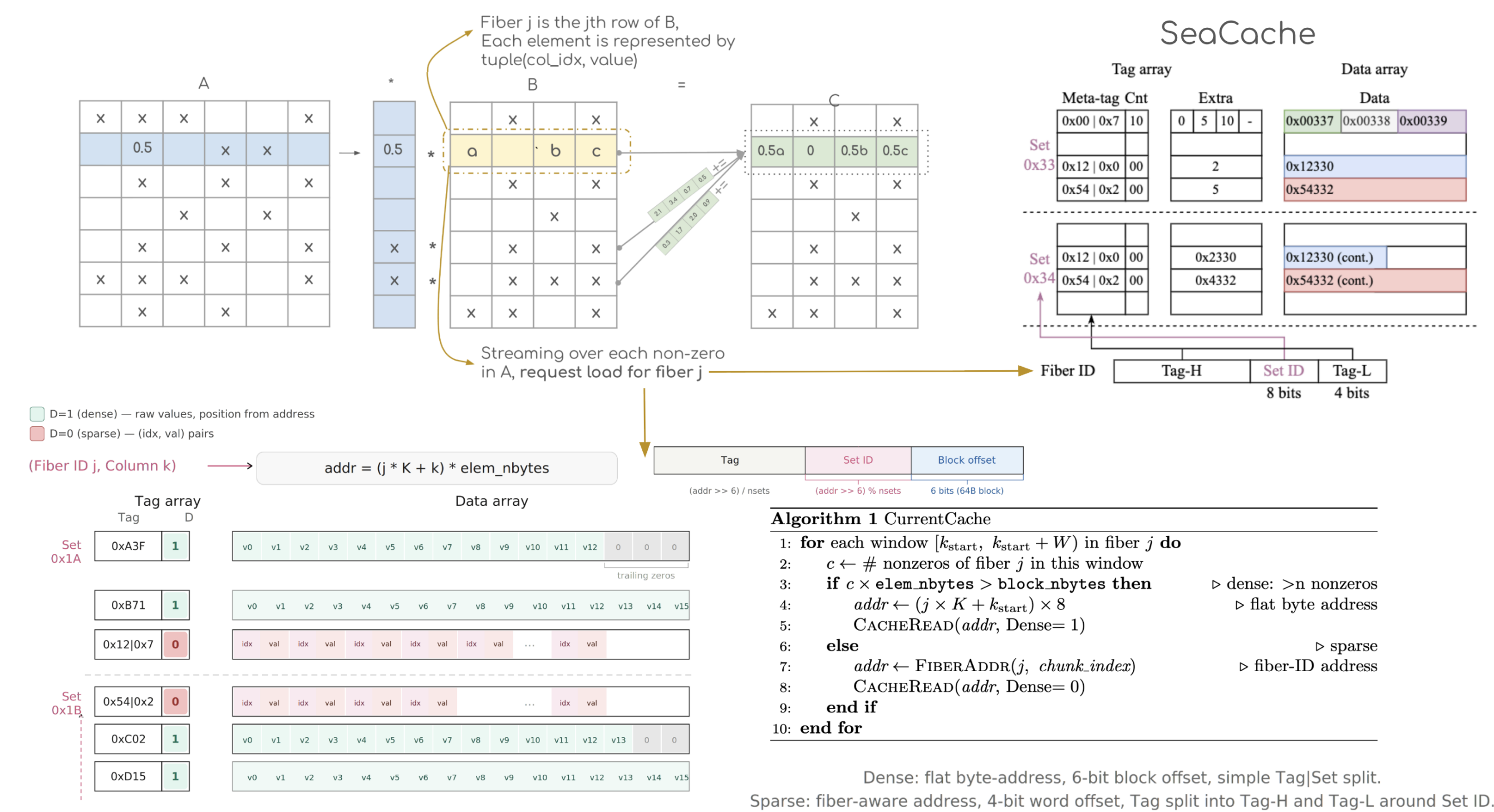


Figure 1. CurrentCache overview.

CurrentCache Design

We present CurrentCache, a hybrid cache controller that combines the SeaCache DSC and a simple dense cache controller. When CurrentCache misses, CurrentCache uses the SeaCache prefetch engine to fetch a number of data blocks, then analyzes the data in each block sequentially using both the SeaCache fiber plus meta-tag and the simple dense format (faithfully representing the padding between nonzero elements) to count which method will install more elements into the cache. CurrentCache then picks that method, calculates the appropriate address, and completes the install using a simple LFU replacement algorithm for the dense cache, and SeaCache's adaptive LFU replacement for a DSC install. All data will therefore be stored in CurrentCache once and at most once after any given load request. CurrentCache must check both the SeaCache index and the dense cache index to calculate a hit detection. This can be done in parallel or sequentially; the tradeoff in read ports and the variance in read overhead must be accounted for by designers.

We also present a methodology to explore in future work for combining more DSCs into a single cache. When benchmarking CurrentCache, we noted that nearly all traces were heavily biased towards either SeaCache or the simple dense cache. Although we can create synthetic data that uses both controllers approximately evenly, real data does not reflect that pattern. Therefore, we propose a fixed-length policy for scaling beyond two DSCs. The first K slots in the policy are sorted by MRU, and each read request is handled in MRU order. The last N slots in the policy are handled by a round-robin selection of the remaining DSCs. Any DSC not in the K + N slots is "idle" and must wait for its chance in the round robin. Each of the K + N DSCs participates in both installs and hit detection, and either a successful install using our data compression heuristic or a successful cache hit will count as a "use" for the multiple-DSC policy. Prefetching may be conducted by majority vote, largest request, MRU or MFU.

Results

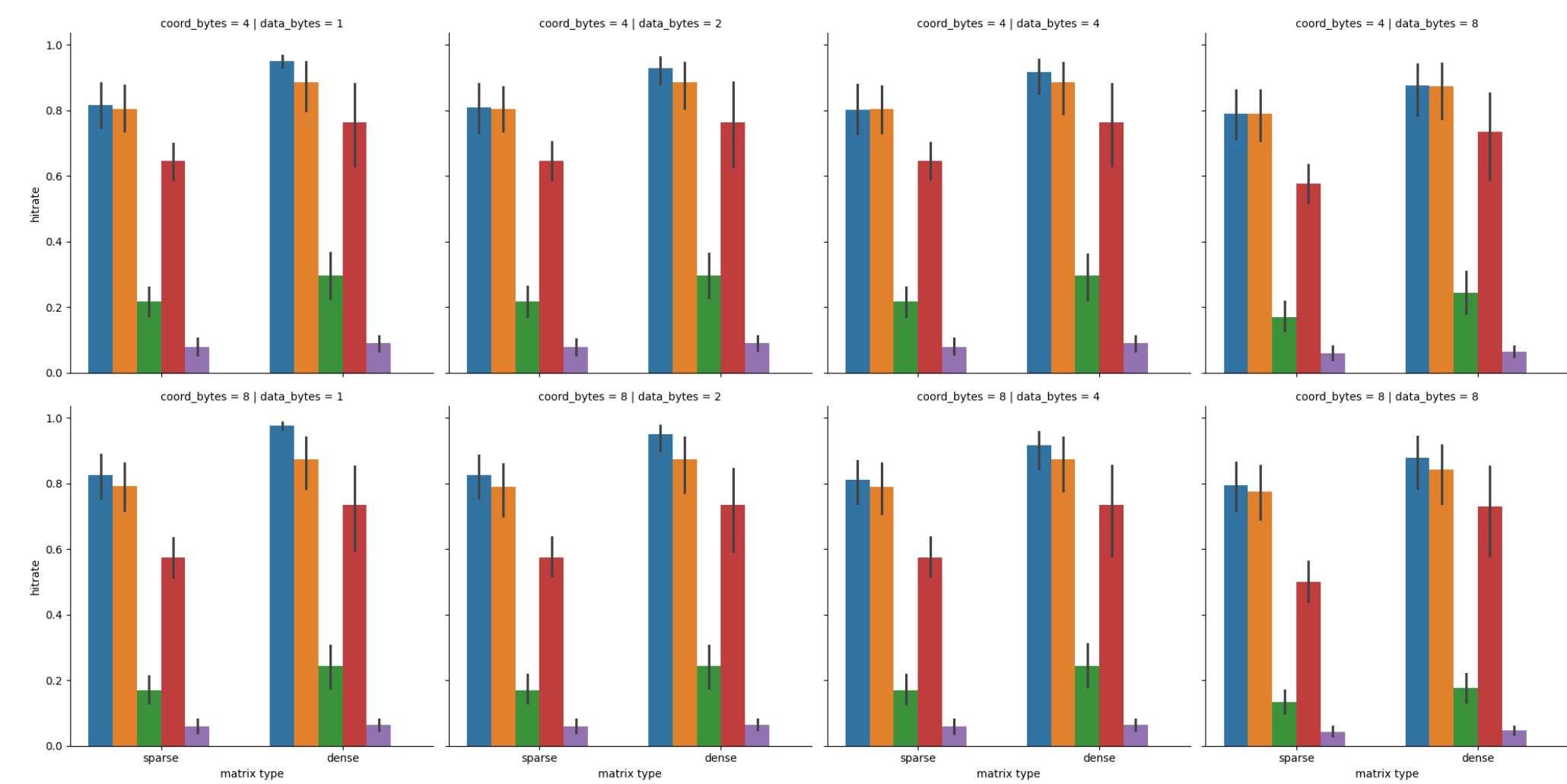


Figure 2. Hitrate over our full suite. Larger is better.

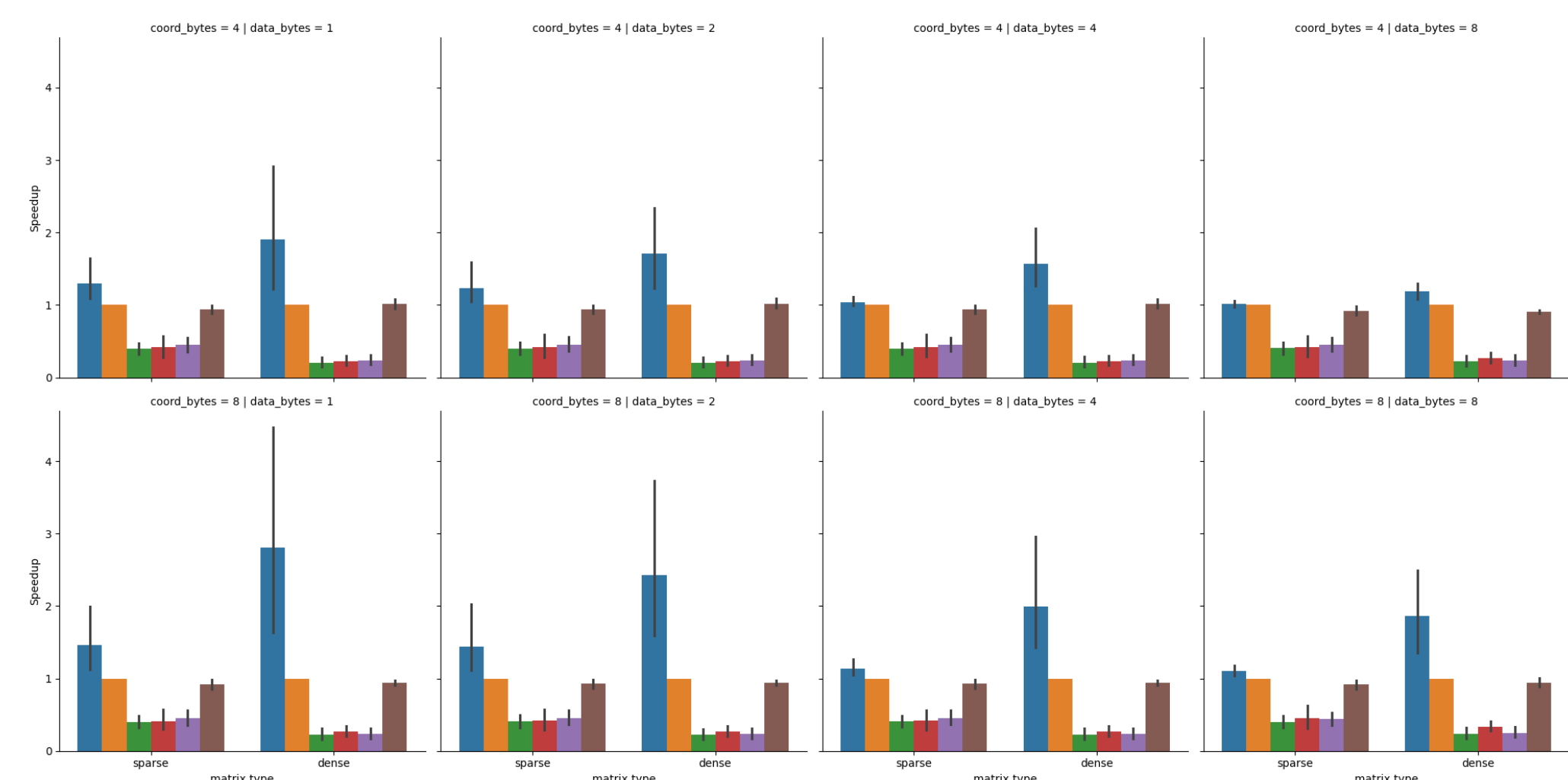


Figure 3. Speedup in cycles per test, normalized to SeaCache. Larger is better.

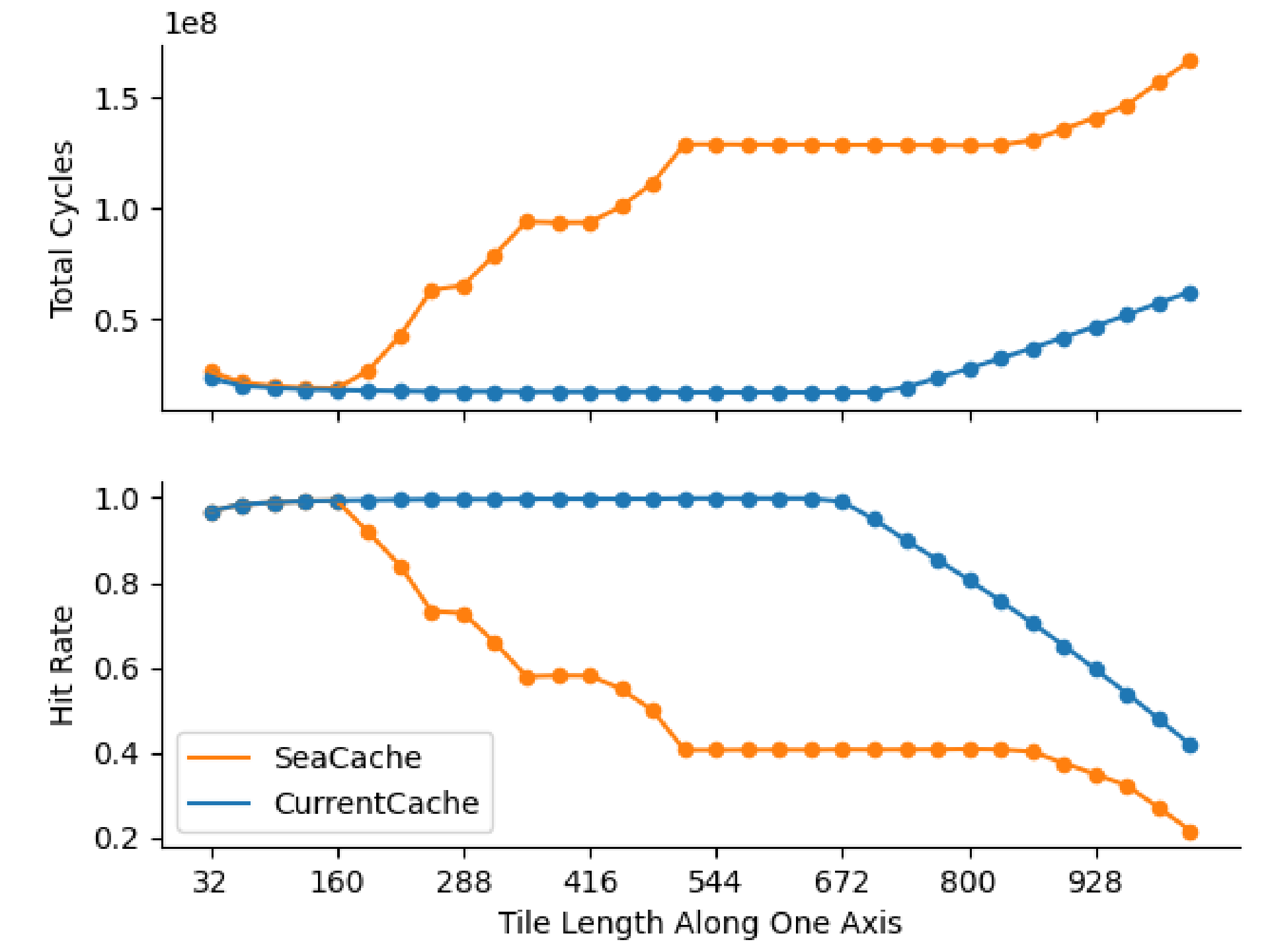


Figure 4. Detailed sweep on dense matrix tilings.

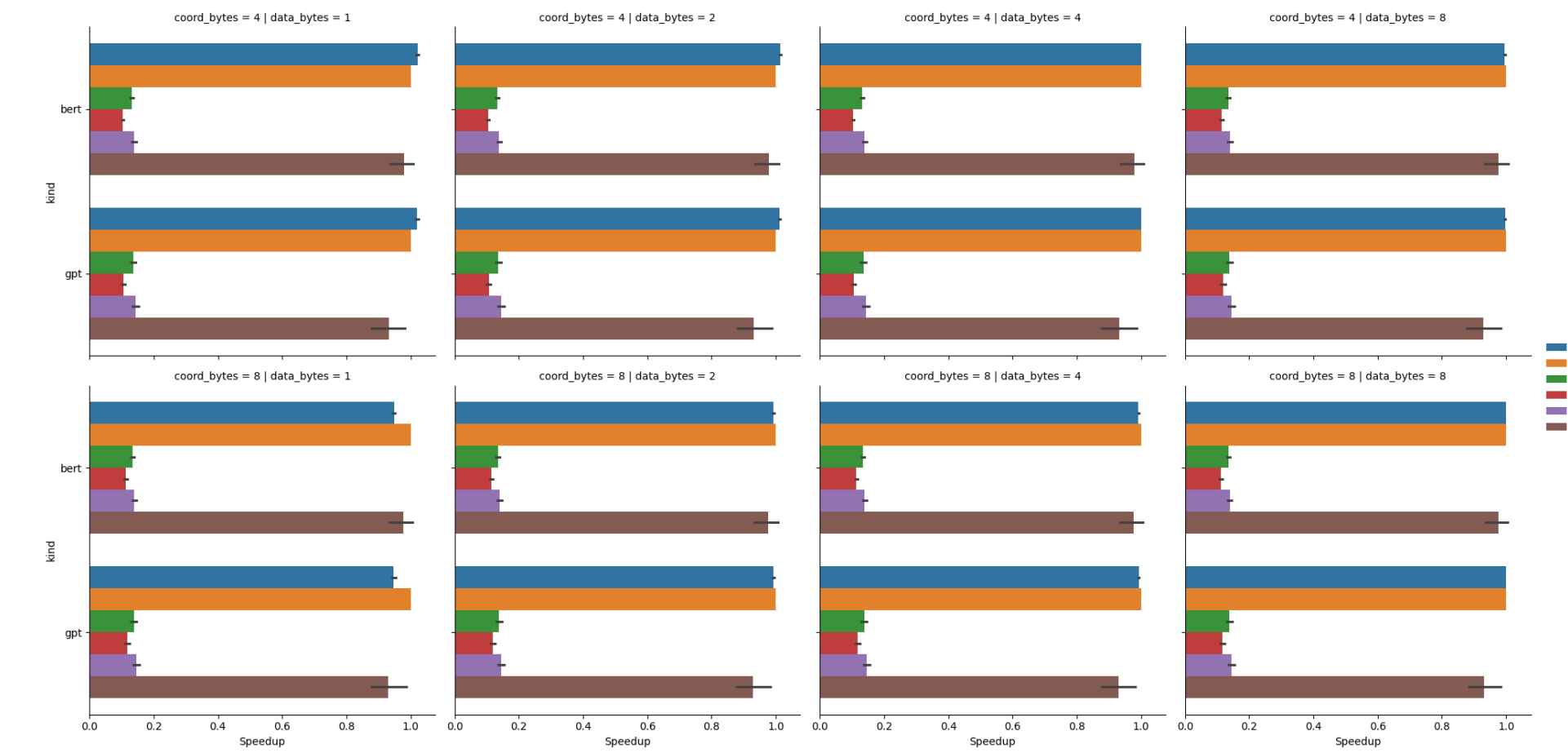


Figure 5. Detailed sweep over GPT & BERT weight matrices. Larger is better.

Methodology and Conclusions

We conducted a preliminary sweep of different paramters to identify interesting sections of the configuration space. We discarded cache size, the number of cache sets, and memory bandwidth since their effect is uniform across our test suite of DSCs, then conducted a grid search over the remaining parameters.

CurrentCache achieves SeaCache or better performance for SpGEMM under nearly all configurations, and strictly improved performance on dense GEMM once the data size eclipses the cache size. This demonstrates the microarchitectural value of smooth, transparent switching between DSCs and a generic dense cache.

References

- [1] Timothy A. Davis and Yifan Hu. The university of florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1), December 2011.
- [2] Xintong Li, Jincheng Jiang, and Mingyu Gao. Seacache: Efficient and adaptive caching for sparse accelerators. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, MICRO '25, page 944–957, New York, NY, USA, 2025. Association for Computing Machinery.
- [3] Riley Murray, James Demmel, Michael W. Mahoney, N. Benjamin Erichson, Maksim Melnichenko, Osman Asif Malik, Laura Grigori, Piotr Luszczek, Michał Dereziński, Miles E. Lopes, Tianyu Liang, Hengrui Luo, and Jack Dongarra. Randomized numerical linear algebra : A perspective on the field with an eye to software, 2023.
- [4] Ali Sedaghati, Milad Hakimi, Reza Hojabr, and Arvindh Shriraman. X-cache: a modular architecture for domain-specific caches. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, page 396–409, New York, NY, USA, 2022. Association for Computing Machinery.
- [5] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, January 2014.